

LAPORAN PRAKTIKUM PBO 3

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Nurfiah S.ST, M.KOM

Asisten Pratikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

Kata Pengantar

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah PBO.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah, M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

Daftar Isi

Kata Pengantar 2

Daftar Isi 3

BAB I 4

BAB II 5

BAB III 7

BAB IV 9

BAB I

Pendahuluan

1. Latar Belakang

Pemrograman Berorientasi Objek (PBO) memungkinkan suatu sistem direpresentasikan melalui objek-objek yang saling berhubungan. Pada praktikum ini, konsep tersebut diterapkan melalui relasi antarobjek menggunakan Class Diagram dan Asosiasi one-to-many. Sistem perbankan yang sebelumnya hanya melakukan transaksi setor dan tarik saldo dikembangkan agar setiap aktivitas transaksi dapat direpresentasikan sebagai objek Transaksi dan disimpan di dalam objek Rekening.

Penerapan relasi tersebut menggunakan struktur data dinamis ArrayList untuk mengelola kumpulan objek transaksi yang dimiliki oleh sebuah rekening. Dengan demikian, sistem tidak hanya dapat melakukan transaksi, tetapi juga menyimpan dan menampilkan riwayat transaksi yang telah dilakukan.

2. Tujuan

- Memahami dan menerjemahkan Class Diagram ke dalam implementasi program menggunakan bahasa Java.
- Memahami serta menerapkan konsep relasi Asosiasi one-to-many antarobjek.
- Mengimplementasikan pengelolaan kumpulan objek menggunakan ArrayList serta menerapkan pencatatan dan penampilan riwayat transaksi.

3. Manfaat

- Meningkatkan pemahaman mengenai hubungan antarclass dan objek dalam Pemrograman Berorientasi Objek.
- Melatih penggunaan ArrayList untuk menyimpan dan mengelola kumpulan objek secara dinamis.
- Memahami penerapan pencatatan riwayat transaksi sehingga aktivitas setor dan tarik tunai dapat direkam serta ditampilkan kembali kepada pengguna.

BAB II

Isi

1. Mengunci Entitas Rekening dan Menambahkan pin

```
public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    double saldo;
    private String pin;

    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String norek, String nama, double saldo, String
pinAwal) {
        this.nomorRekening = norek;
        this.namaPemilik = nama;
        this.saldo = saldo;

        if(pinAwal.length() == 6) {
            this.pin = pinAwal;
        }
        else {
            System.out.println("Pin harus 6 digit");
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama : " + namaPemilik + "
Berhasil dibuat.");
    }

    public String getNomorRekening() {
        return this.nomorRekening;
    }

    public String getNamaPemilik() {
        return this.namaPemilik;
    }

    public boolean autentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
}
```

Kode ini mendefinisikan kelas Rekening yang merepresentasikan rekening bank sederhana, dengan atribut nomor rekening, nama pemilik, saldo, PIN, dan daftar riwayatTransaksi bertipe ArrayList<Transaksi>. Konstruktornya mengisi semua atribut tersebut, memvalidasi bahwa PIN harus 6 karakter (jika tidak, hanya mencetak pesan error dan pin tetap null), lalu membuat list riwayat transaksi kosong dan mencetak pesan bahwa rekening berhasil dibuat. Ada dua getter (getNomorRekening dan getNamaPemilik) serta method autentikasi yang membandingkan PIN

input dengan PIN tersimpan menggunakan equals. Perlu diperhatikan, jika PIN awal tidak valid maka pin bernilai null, sehingga pemanggilan autentikasi akan menyebabkan NullPointerException; selain itu kode ini butuh import java.util.ArrayList dan kurung kurawal penutup kelasnya belum ada.

2. Mengunci Entitas Transaksi

```
public class Transaksi {  
    private String idTransaksi;  
    private String jenis;  
    private double nominal;  
  
    public Transaksi(String id, String jenis, double nom) {  
        this.idTransaksi = id;  
        this.jenis = jenis;  
        this.nominal = nom;  
    }  
  
    public String getIdTransaksi() {  
        return this.idTransaksi;  
    }  
  
    public String getJenis() {  
        return this.jenis;  
    }  
  
    public double getNominal() {  
        return this.nominal;  
    }  
}
```

Kode ini mendefinisikan kelas Transaksi yang merepresentasikan satu catatan transaksi, dengan tiga atribut privat: idTransaksi, jenis (misalnya setor atau tarik), dan nominal bertipe double. Konstruktornya menerima ketiga nilai tersebut lalu menyimpannya ke atribut, dan tersedia tiga getter (getIdTransaksi, getJenis, getNominal) untuk membacanya dari luar kelas. Karena tidak ada setter, objek ini bersifat read-only setelah dibuat, cocok untuk riwayat transaksi di kelas Rekening yang menyimpannya dalam ArrayList<Transaksi>. Sama seperti sebelumnya, kurung kurawal penutup kelasnya belum ada.

BAB III

Latihan

1. Menu 1 (Buka Rekening)

```
switch(pilihan1) {  
    case 1:  
        System.out.println("Masukkan PIN awal:");  
        String pin = input.nextLine();  
        System.out.println("Masukkan no rekening: ");  
        String no = input.nextLine();  
        System.out.println("Masukkan nama pemilik: ");  
        String nama = input.nextLine();  
        System.out.println("Masukkan saldo awal: ");  
        double saldo = input.nextDouble();  
  
        if (saldo >= 50000) {  
            akunAktif = new Rekening(no,nama, saldo, pin);  
            rekAr.add(akunAktif);  
        }  
}
```

Kode ini adalah case 1 (Buka Rekening) di Main.java: program meminta PIN awal, nomor rekening, nama pemilik, dan saldo awal lewat Scanner, lalu jika saldo minimal 50.000, dibuat objek Rekening baru (konstrukturnya memvalidasi PIN 6 digit), disimpan ke akunAktif sebagai akun yang sedang dipakai, dan ditambahkan ke list rekAr. Kelemahannya, tidak ada else sehingga user tidak diberi tahu bila saldo kurang, akunAktif tetap menunjuk akun lama dalam kasus itu, dan nextDouble() meninggalkan newline di buffer sehingga perlu input.nextLine() tambahan agar input berikutnya tidak terlewat.

2. Keamanan Transaksi

```
case 2:  
    System.out.println("Masukkan PIN anda");  
    String pn = input.nextLine();  
    System.out.println("Masukkan nominal tarik: ");  
    double tarik = input.nextDouble();  
    if (akunAktif.autentikasi(pn)) {  
        akunAktif.tarikTunai(tarik);  
    }  
  
    else {  
        System.out.println("PIN ANDA SALAH");  
    }  
    break;
```

```

case 4:
    System.out.println("Masukkan PIN anda");
    String pn2 = input.nextLine();

    if (akunAktif.autentikasi(pn2)) {
        akunAktif.CetakMutasi();
    }

    else {
        System.out.println("PIN ANDA SALAH");
    }

    break;

```

Kode ini berisi dua case yang menerapkan otentikasi PIN: case 2 (Tarik Tunai) meminta PIN dan nominal tarik, lalu jika autentikasi(pn) bernilai true memanggil tarikTunai(tarik), sedangkan case 4 (Cetak Mutasi) meminta PIN lalu memanggil CetakMutasi() hanya jika autentikasi(pn2) benar; jika PIN salah, keduanya mencetak "PIN ANDA SALAH". Ini sudah sesuai konsep tugas, tetapi ada beberapa catatan: pesan error belum sama dengan yang diminta modul ("Akses Ditolak: PIN yang Anda masukkan salah!"), nominal di case 2 diminta sebelum PIN diverifikasi sehingga urutannya kurang tepat, nextDouble() meninggalkan newline di buffer yang bisa membuat nextLine() berikutnya terlewat, dan akunAktif yang masih null (belum ada rekening dibuat) akan menyebabkan NullPointerException.

BAB IV

Kesimpulan

1. Kesimpulan

Praktikum 3 ini menunjukkan bahwa enkapsulasi merupakan fondasi keamanan dan integritas data dalam pemrograman berorientasi objek. Dengan mengubah atribut kelas Transaksi dan Rekening menjadi private, data sensitif seperti saldo dan PIN tidak lagi bisa dimanipulasi langsung dari luar kelas (misalnya `akunAktif.saldo = 9999999`), melainkan hanya lewat method public yang memuat logika validasi. Kelas Transaksi dibuat immutable dengan hanya menyediakan getter tanpa setter, sedangkan Rekening sengaja tidak menyediakan `setSaldo()` dan `getPin()` agar perubahan saldo hanya terjadi melalui transaksi resmi (setor dan tarik) dan PIN tidak pernah terekspos. Validasi panjang PIN 6 digit pada konstruktor serta method `otentikasi()` menjadi lapisan keamanan yang kemudian diterapkan pada menu Tarik Tunai dan Cetak Mutasi di `Main.java`. Dari praktikum ini dapat disimpulkan bahwa penggunaan access modifier yang tepat, pembatasan getter dan setter, serta penyisipan validasi bisnis di dalam kelas mampu menjaga konsistensi data sekaligus membuat sistem lebih aman dan mudah dirawat.